

Final Exam

Version: <Sample>

Academic Integrity

- *Aggies do not lie, cheat, or steal, nor tolerate those who do.*
- We hope you have fun solving these problems, even though this is an exam ☺
- ***You can do this!***

Exam Guidelines

- You will have 120 minutes to complete and submit this exam.
- Do not write outside of the margin box. We scan the exams and anything outside those lines will likely get cut off.
- You cannot use any electronic devices (including calculators, phones, smart watches, and computers)
- You may use
 - A writing utensil (e.g. pen/pencil)
 - Scratch Paper that we provide.
 - § Scratch paper may not have the margin box, so leave a margin around the edge of your scratch paper to avoid having information cut off when scanned.
 - Up to 5 pages of exam aids.
 - § Exam aids can be pages up to 8.5x11 inches and can be handwritten or printed on both sides.
 - § Do not use exam aids for scratch work.
 - If you use an exam aid for scratch work, submit it with the exam.
- Start Exam when prompted.
- Stop the exam when you are finished or when time expires.
 - Attach any scratch work to the end of your exam. Including exam aids used for scratch work.
 - Submit your exam.
 - § You may keep your exam aids if they were not used for scratch work.

Fill in name and UIN before exam starts

Name:_____

UIN:_____

Q1 - Code Tracing (42 points)

Overview

- In the following parts you will be given small pieces of code. Determine what each piece of code prints, and fill in the blank.
- For each piece of code, assume that the file has `#include<iostream>, #include<string>, #include<vector>, #include<climits>, #include<sstream>, #include<stdexcept>` and using `std::cout, std::string, std::vector, std::ostream, std::stringstream, std::invalid_argument;`
- Unless a part specifically says otherwise, **you may assume that all of the code snippets successfully compile**, will not throw uncaught exceptions, and will not have undefined behavior

Q1.1

```
class string_box {
    string* data;
public:
    string_box(string s): data{new string{s}} {}
    string_box(const string_box& source) :
        data{new string{*(source.data)}} {}
    void change_string(string s) {
        *data = s;
    }
    void print() {
        cout << *data;
    }
    ~string_box() {
        delete data;
    }
};

int main() {
    string_box a("A");
    string_box* b = new string_box("B");
    *b = a;
    a.change_string("C");
    a.print();
    b->print();
}
```

This code prints: _____

Does this code have a memory leak?

- ☐ Yes
- ☐ No

Q1.2

```
class foo {
public:
    int x;
    foo (int val): x{val} {}
    int fun() { return x; };
};

class bar : public foo {
public:
    int y;
    bar (int val): foo(val), y{2} {}
    int fun() { return y; }
};

int main() {
    bar A(9);
    cout << A.fun() << A.foo::fun();
}
```

This code prints: _____

Q1.3

```
template <typename T>
class foo {
public:
    T x;
    foo (T val): x{val} {}
    void func(T val1) {
        x = val1;
    }
};

int main() {
    vector<foo<char>> v;
    v.push_back('a');
    v.push_back('b');
    v.at(0).func('c');
    cout << v.at(0).x << v.at(1).x;
}
```

This code prints: _____

Q1.4

```
class Num {
public:
    int x, y;
    Num(int a, int b) : x{a}, y {b} {}

    Num operator+(const Num& other) const {
        return Num(x + other.x, y);
    }
};

ostream& operator<<(ostream& os, const Num& n) {
    os << n.x << n.y;
    return os;
}

int main() {
    Num a(2,1);
    Num b(5,6);
    Num c = a + b;
    cout << c;
}
```

This code prints: _____

Q1.5

```
void adjust(int* x, int& y) {
    x[2] = 5;
    x = new int[4]{14,11,16,19};
    y = x[1];
}

int main() {
    int size = 3;
    int* arr = new int[size]{18, 12, 16};
    adjust(arr, size);
    cout << size << arr[2];
    delete[] arr;
}
```

This code prints: _____

Does this code have a memory leak?

- ☐ Yes
- ☐ No

Q1.6

```
int mystery(int x) {
    if (x > 8) {
        return x;
    }
    return mystery(x*2);
}

int main() {
    cout << mystery(1);
}
```

This code prints: _____

Q1.7

```
double func(double A) {
    cout << "joe ";
    return A * 2.0;
}

double func(int A) {
    cout << "bob ";
    return A + 2.0;
}

int main() {
    cout << func(func(15));
}
```

This code prints: _____

Q1.8

```
void func(vector<int>& p, vector<int> q) {
    p = q;
    q[1] = 14;
}

int main() {
    vector<int> v1{10, 13, 15};
    vector<int> v2{1, 2};
    func(v1, v2);
    cout << v1.at(0) << v1.at(1) << v2.at(0) << v2.at(1);
    return 0;
}
```

This code prints: _____

Q1.9

```
class A {
    int* x;
public:
    A(int val) : x{new int{val}} {}
    virtual void func() {
        cout << *x;
    }
};
class B : public A {
    int* y;
public:
    B(int xVal, int yVal) : A(xVal), y{new int{yVal}} {}
    void func() override {
        cout << *y;
    }
};
int main() {
    A* ptr = new B(2,3);
    ptr->func();
}
```

This code prints: _____

Q1.10

```
void f(int& a, int* b) {
    int* c = new int{3};
    a = *c;
    b = c;
    delete c;
}

int main() {
    int x = 1;
    int* y = new int{2};
    f(x, y);
    cout << x << *y;
    delete y;
}
```

This code prints: _____

Does this code have a memory leak?

☐ Yes☐ No

Q1.11

```

class animal {
public:
    int w = 1;
protected:
    double x = 2.0;
};
class bear : public animal {
private:
    long long y = 5;
};
class panda : public bear {
public:
    unsigned int z = 7;
    void func(){
        cout << z;
    }
};
int main() {
    panda p;
    p.func();
    cout << p.z;
}

```

Fill in the following table with Yes/No to indicate which attributes of the panda class are accessible in panda.func (i.e. which attributes could validly replace z in the print statement in func), and which attributes of p are accessible in main (i.e. which attributes could validly replace z in the print statement in main).

	w	x	y	z
Accessible in func				Yes
Accessible in main				Yes

Q1.12

```

int main() {
    stringstream ss("11 * 12 + 13");
    int x = 0;
    int sum = 0;
    while(ss >> x) {
        sum += x;
    }
    cout << sum;
}

```

This code prints: _____

Q1.13

```

struct int_box {
    int* x;
};
class int_box_box {
    int_box* p;
public:
    int_box_box(int& q): p{new int_box{new int{q}}} {}
};
int main() {
    int a = 7;
    int_box* b = new int_box{&a};
    int_box_box bb(a);
    // questions are about this point in the code
}

```

Fill the table to identify where the value is stored *when the code reaches the commented line*, i.e. fill the table with either Stack or Heap:

	a	b->x	*(b->x)	bb.p	bb.p->x	*(bb.p->x)
Stack/Heap						

Q1.14

```

int sum(int x, int y) {
    if (x > INT_MAX - y) {
        throw invalid_argument("Overflow!");
    }
    return x + y;
}

int main() {
    cout << sum(INT_MAX, 1 - INT_MAX);
}

```

Note that INT_MAX is the maximum value that can be stored in a signed int. What will be the output of this code? Choose the best answer.

- ☐ 1
- ☐ The program will throw an invalid_argument exception
- ☐ The program will throw an overflow_error exception
- ☐ The program has undefined behavior

Q2 - Swap Even Values (29 points)

Overview

Write a function which swaps nodes with even values between two linked lists until one linked list runs out of nodes with even values

Requirements

Write the function `void linked_list::swap_evens(linked_list& other)` which swaps nodes with even values from the list with nodes with even values from `other` until one of the lists runs out of nodes with even values.

- Note that we are swapping nodes storing even **values**, not nodes at even indices
- The first even-valued node of the list should swap with the first even-valued node of `other`, the second even-valued node should swap with the second even-valued node of `other`, and so on.
 - i.e. do not change the relative order of the even-valued nodes.
- Note that `swap_evens` is a member function of the `linked_list` class
- Both the `linked_list` calling `swap_evens` and the `linked_list` `other` should be **modified** by the `swap_evens` method
- You must not have any memory errors or leaks
- For full credit you **should NOT** allocate any dynamic memory.
- For full credit you should not use any libraries

Assume the following definition for `linked_list` (you should not assume that the class has any functions besides the ones listed here):

```
struct node {
    int data;
    node* next;
    node(int data, node* next) : data(data), next(next) {}
    // Node struct uses the default destructor, copy constructor,
    // and copy assignment operator
};

class linked_list {
private:
    node* head;
public:
    linked_list() : head(nullptr) {}
    // Assume the rule of 3 functions have been implemented correctly
    ~linked_list();
    linked_list(const linked_list& other);
    linked_list& operator=(const linked_list& other);
    // Write this function
    void swap_evens(linked_list& other);
};
```

Examples

- **Before calling swap_evens:**
 - **Linked list list1:**
 - 1 -> 2 -> 3 -> 4 -> 5 -> 6
 - **Linked list list2:**
 - 12 -> 11 -> 10 -> 9 -> 8 -> 7
- **After calling list1.swap_evens(list2):**
 - **list1:**
 - 1 -> 12 -> 3 -> 10 -> 5 -> 8
 - **list2:**
 - 2 -> 11 -> 4 -> 9 -> 6 -> 7

- **Before calling swap_evens:**
 - **Linked list list1:**
 - 1 -> 1 -> 3 -> 6 -> 1 -> 9
 - **Linked list list2:**
 - 1 -> 4 -> 16 -> 256 -> 65536
- **After calling list1.swap_evens(list2):**
 - **list1:**
 - 1 -> 1 -> 3 -> 4 -> 1 -> 9
 - **list2:**
 - 1 -> 6 -> 16 -> 256 -> 65536

- **Before calling swap_evens:**
 - **Linked list list1:**
 - 1 -> 3 -> 2 -> 2 -> 2 -> 11
 - **Linked list list2:**
 - 5 -> 4 -> 5 -> 4 -> 5
- **After calling list1.swap_evens(list2):**
 - **list1:**
 - 1 -> 3 -> 4 -> 4 -> 2 -> 11
 - **list2:**
 - 5 -> 2 -> 5 -> 2 -> 5

- **Before calling swap_evens:**
 - **Linked list list1:**
 - 1 -> 3 -> 2 -> 2 -> 2 -> 11
 - **Linked list list2 is empty**
- **After calling list1.swap_evens(list2):**
 - Both lists are unchanged

Name:_____

11 / 16

Q2 Answer (Swap Even Values)

```
void linked_list::swap_evens(linked_list& other)
```

Name: _____

12 / 16

Q2 Answer (continued)

Q3 - Struct Processing (29 points)

Overview

Write a function which creates a rectangular 2-D vector where each row consists of a list of integers in ascending order. Use a provided vector of structs to determine which integers to use in each row.

Requirements

Write the function `vector<vector<int>> process_structs(const vector<bounds>& input)` which takes as input a vector of bounds objects, each of which contains attributes `min` and `max`, and uses it to return a 2-D vector in the following way:

- For each bounds object with `min <= max`, add a row to the 2-D vector where the elements are the integers from `min` to `max` (inclusive), increasing in increments of 1
- Skip each bounds object with `min > max` (do not add a new row to the output vector)
- After processing all elements of the input vector, trim each row to the same length as the shortest row by removing elements from the end, so that the resulting 2-D vector is rectangular

Remarks:

- The rows of the 2-D vector should be in the same order as the bounds objects in the input vector
- For full credit you should not use any libraries besides `<vector>`

Assume the following definition for the `bounds` struct:

```
struct bounds {  
    int min;  
    int max;  
};
```

Examples

- Input Vector: `{{1, 6}, {4, 1}, {3, 4}, {2, 5}}`
- 2-D vector before trimming:
`{{1, 2, 3, 4, 5, 6},
 {3, 4},
 {2, 3, 4, 5}}`
- Vector returned by `process_structs`:
`{{1, 2},
 {3, 4},
 {2, 3}}`
- Input Vector: `{{3, 100}, {90, 89}, {5, 5}, {7, -2}, {-4, -2}}`
- Vector returned by `process_structs`:
`{{3},
 {5},
 {-4}}`

Q3 Answer (Struct Processing)

```
#include<vector>
```

```
using std::vector;
```

```
vector<vector<int>> process_structs(const vector<bounds>& input)
```

Name: _____

15 / 16

Q3 Answer (continued)

Name: _____

16 / 16